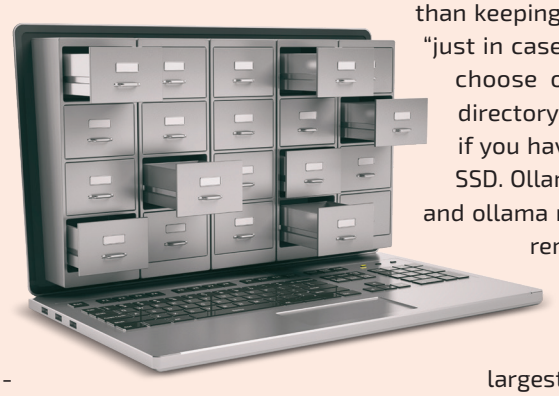
MODEL STORAGE IS THE SILENT TAX

Local AI has no per-message subscription meter, but it can quietly develop another bill: disk space. One 4-bit 4B model may occupy only a few gigabytes. Move to 8-bit, larger parameter counts, vision models and several competing families, and a 512GB laptop can start feeling small surprisingly quickly.

Duplication makes the problem worse. LM Studio, Ollama and a manually managed llama.cpp folder can each maintain their own model libraries. Downloading the same underlying weights through three tools for testing may therefore consume multiple copies rather than one magical shared file. Add old quantisations that you forgot about and storage disappears in chunks of 5, 10 or 20GB.



Adopt a monthly model audit. Keep a simple note containing model name, quantisation, runtime, purpose and file size. Delete superseded variants rather than keeping Q4, Q5, Q8 and FP16 "just in case". LM Studio lets you choose or change its model directory, which can be useful if you have a larger secondary SSD. Ollama provides ollama ls and ollama rm for inspecting and removing its models.

The smartest local-model collection is not the largest one. It is the smallest set that covers your real workloads: perhaps one fast everyday model, one stronger model and one specialist.

Route 3: llama.cpp – take the training wheels off

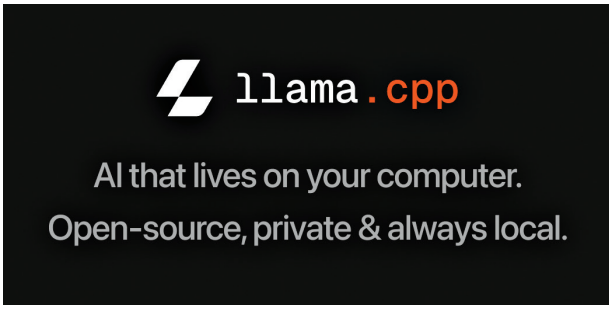
llama.cpp is a C/C++ inference project whose goal is to run LLMs and VLMs efficiently across a very wide range of hardware. It supports CPU inference, multiple GPU backends, quantised GGUF models and CPU/GPU hybrid execution. If LM Studio is the polished dashboard and Ollama the convenient terminal appliance, llama.cpp lets you see more of the engine without requiring you to compile it yourself.

Step 1: Install a current build

The llama.cpp project offers multiple installation paths, including pre-built binaries, package managers, Docker and building from source. On Windows, one straightforward current route is:

```
winget install llama.cpp
```

On macOS/Linux, use the project's current installation guidance; advanced users can also download release binaries or build from source. After installation, check the version or help output to confirm the command name provided by your package.



Step 2: Download and run directly from Hugging Face

Current llama.cpp can fetch a compatible model from Hugging Face directly. Run:

```
llama cli -hf ggml-org/Qwen3-4B-GGUF:Q4_K_M
```

The `-hf` option specifies a Hugging Face repository and optional quantisation. llama.cpp's CLI currently defaults to Q4_K_M when a repository is supplied without a quantisation and that variant exists, but writing `:Q4_K_M` explicitly makes your experiment reproducible.

Paste the same battery prompt.

You have now run the same underlying GGUF through three different front ends or runtime workflows.

Step 3: Set context deliberately

Exit and run it again with an explicit context:

```
llama cli -hf ggml-org/Qwen3-4B-GGUF:Q4_K_M
-c 4096
```

The `-c` or `--ctx-size` argument controls context size. Starting at 4096 keeps memory use sensible for our test.

Do not interpret this as the model's maximum capability. We are choosing a working value appropriate to the task.

Step 4: Experiment with GPU offload

llama.cpp exposes `-ngl`, or `--n-gpu-layers`, to control